

Today 11:12 AM

=

Hello, could you please help me  [Free offer](#)   ...

number guessing game? When I click SUBMIT with an empty text box, submit_guess() in game_engine.py crashes with ValueError: invalid literal for int() with base 10: ''. Could you add validation so that empty input never reaches int(), the attempt count is preserved, and self.feedback_msg asks the player to enter a valid number first? Here is my current code:

```
import random
import pygame
from game.text_box import TextBox
```

[Show more](#) ▾

Yes. The key is to validate the text **before** calling `int()`. Invalid/empty input should return early, so `attempts` stays unchanged and the text box is not unnecessarily cleared.

Replace your `submit_guess()` with:

</> Python



▶ Run

```
def submit_guess(self):
    if self.game_won:
        return

    # Validate input before converting to int
    text = self.input_box.text.strip()

    if not text:
        self.feedback_msg = "Please enter a valid number first."
        self.feedback_color = (240, 200, 80)
        return

    try:
        guess = int(text)
    except ValueError:
        self.feedback_msg = "Please enter a valid number first."
        self.feedback_color = (240, 200, 80)
        return
```

```

self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
    self.feedback_msg = f"T00 LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)
elif guess > self.secret_number:
    self.feedback_msg = f"T00 HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)
else:
    self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
    self.feedback_color = (80, 220, 90)
    self.game_won = True

```

What this fixes

- **Empty input:** " " or "" → feedback asks for a valid number.
- **Non-numeric input:** "abc" → same feedback instead of crashing.
- **Attempts:** invalid input **does not increment** `self.attempts`.
- **Input box:** invalid input is **not cleared**, allowing the player to correct it.
- **Valid input:** works exactly as before.
- **Game won:** still prevents further submissions.

For example:

Attempts: 0
[] [SUBMIT]



Player clicks SUBMIT with empty box

Attempts: 0
[] [SUBMIT]
Please enter a valid number first.

Then if they enter 50 :

Attempts: 1
T00 LOW! (Guess was 50)



This is preferable to simply doing `if self.input_box.text:` because the `try/except` also protects against other invalid strings reaching `int()`.



Absolutely. We can keep this very small: add two variables, update them only after a **valid** guess, reset them in `reset()`, and render the range.

1. Add the bounds in `__init__`

Right after `self.attempts = 0`:

</> Python



▶ Run

```
self.low_bound = 1
self.high_bound = 100
```

2. Update the bounds in `submit_guess()`

Use this version of the relevant section:

</> Python



▶ Run

```
def submit_guess(self):
    if self.game_won:
        return

    text = self.input_box.text.strip()

    if not text:
        self.feedback_msg = "Please enter a valid number first."
        self.feedback_color = (240, 200, 80)
        return

    try:
        guess = int(text)
    except ValueError:
        self.feedback_msg = "Please enter a valid number first."
        self.feedback_color = (240, 200, 80)
        return

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.low_bound = max(self.low_bound, guess + 1)
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
```

```

elif guess > self.secret_number:
    self.high_bound = min(self.high_bound, guess - 1)
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)

else:
    self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
    self.feedback_color = (80, 220, 90)
    self.game_won = True

```

3. Reset the bounds

Add these two lines inside `reset()` :

</> Python



▶ Run

```

self.low_bound = 1
self.high_bound = 100

```

So your reset becomes:

</> Python



▶ Run

```

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.low_bound = 1
    self.high_bound = 100
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.input_box.clear()

```

4. Show the range on screen

In `render()` , after the attempts text:

</> Python



▶ Run

```

range_surf = self.font_medium.render(
    f"Current Possible Range: {self.low_bound} - {self.high_bound}",
    True,
    (180, 185, 195)
)
screen.blit(
    range_surf,
    (self.width // 2 - range_surf.get_width() // 2, 125)
)

```

So that section becomes:

</> Python

Run

```
attempts_surf = self.font_medium.render(
    f"Attempts: {self.attempts}", True, (180, 185, 195)
)
screen.blit(
    attempts_surf,
    (self.width // 2 - attempts_surf.get_width() // 2, 95)
)

range_surf = self.font_medium.render(
    f"Current Possible Range: {self.low_bound} - {self.high_bound}",
    True,
    (180, 185, 195)
)
screen.blit(
    range_surf,
    (self.width // 2 - range_surf.get_width() // 2, 125)
)

self.input_box.render(screen)
```

What changed

Change	Behavior
low_bound = 1	Starting minimum
high_bound = 100	Starting maximum
Guess too low	low_bound = guess + 1
Guess too high	high_bound = guess - 1
Invalid/empty input	Bounds don't change
Correct guess	Bounds remain as they are
reset()	Bounds return to 1 - 100
render()	Displays Current Possible Range: X - Y

For example, if the player guesses 23 and gets **TOO LOW**, the range becomes:

Current Possible Range: 24 - 100

Then if they guess 68 and get **TOO HIGH**:

Current Possible Range: 24 – 67



Using `max()` and `min()` also prevents a later guess from accidentally widening the range.



The range hints work. Could you please add a history panel below the feedback box showing the player's last 5 guesses, each with a coloured tag showing whether it was too high or too low? It should clear when the game restarts.

Yes. We can keep this minimal by storing a list of the last 5 valid guesses along with their result, then rendering them below the feedback message.

1. Add `guess_history` in `__init__`

After your range variables:

</> Python



▶ Run

```
self.low_bound = 1
self.high_bound = 100
self.guess_history = []
```

Each entry will store:

</> Python



▶ Run

```
(guess, result, color)
```

2. Add each valid guess to the history

Update the result section of `submit_guess()` :

</> Python



▶ Run

```
if guess < self.secret_number:
    self.low_bound = max(self.low_bound, guess + 1)
    self.feedback_msg = f"T00 LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)
    result = "T00 LOW"
    result_color = (80, 160, 240)

elif guess > self.secret_number:
    self.high_bound = min(self.high_bound, guess - 1)
    self.feedback_msg = f"T00 HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)
    result = "T00 HIGH"
    result_color = (240, 100, 80)

else:
    self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
    self.feedback_color = (80, 220, 90)
    self.game_won = True
    result = "CORRECT"
    result_color = (80, 220, 90)

self.guess_history.append((guess, result, result_color))

# Keep only the latest 5 guesses
if len(self.guess_history) > 5:
    self.guess_history.pop(0)
```

Because this comes **after validation**, invalid/empty submissions won't appear in the history.

3. Clear the history on restart

Add this to `reset()` :

</> Python



▶ Run

```
self.guess_history = []
```

So:

</> Python



▶ Run

```
def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.low_bound = 1
    self.high_bound = 100
    self.guess_history = []
```

```

self.feedback_msg = "Enter a number between 1 and 100"
self.feedback_color = (220, 220, 220)
self.game_won = False
self.input_box.clear()

```

4. Render the history panel

Add this **after the feedback rendering** in `render()` :

</> Python



Run

```

history_title = self.font_medium.render(
    "Guess History", True, (220, 220, 220)
)
screen.blit(
    history_title,
    (self.width // 2 - history_title.get_width() // 2, 275)
)

for i, (guess, result, color) in enumerate(reversed(self.guess_history)):
    y = 315 + i * 32

    guess_text = self.font_medium.render(
        str(guess), True, (245, 245, 245)
    )
    screen.blit(
        guess_text,
        (self.width // 2 - 100, y)
    )

    tag_rect = pygame.Rect(self.width // 2, y, 110, 26)
    pygame.draw.rect(screen, color, tag_rect, border_radius=5)

    tag_text = self.font_btn.render(result, True, (255, 255, 255))
    screen.blit(
        tag_text,
        (
            tag_rect.centerx - tag_text.get_width() // 2,
            tag_rect.centery - tag_text.get_height() // 2
        )
    )

```

Result

You'll get something like:

T00 LOW! (Guess was 35)



Guess History

```
52      [ T00 HIGH ]
41      [ T00 HIGH ]
24      [ T00 LOW  ]
35      [ T00 LOW  ]
43      [ T00 HIGH ]
```

The **most recent guess appears first**, and once there are more than five guesses, the oldest one disappears.

One small detail: I kept the **CORRECT** guess in the history too, since it is still a valid guess. The panel automatically clears when R starts a new game.



Today 6:26 PM

Finally, could you please add a maximum of 7 attempts? If the player runs out without guessing correctly, the game should enter a `GAME_OVER` state that reveals the secret number and prompts them to press R to try again. R should reset everything (bounds, history, attempts). Please also show attempts as "Attempts: 3 / 7".

Yes. The cleanest approach is to add a `max_attempts` constant and a `game_over` state, while keeping your existing `game_won` state.

1. Add the maximum attempts and `game_over`

In `__init__`, change:

</> Python

```
self.attempts = 0
```



▶ Run

to:

</> Python

```
self.attempts = 0
self.max_attempts = 7
self.game_over = False
```



▶ Run

Your state variables will now include:

</> Python



Run

```
self.attempts = 0
self.max_attempts = 7
self.game_over = False
self.game_won = False
```

2. Update submit_guess()

Replace your current `submit_guess()` with this:

</> Python



Run

```
def submit_guess(self):
    if self.game_won or self.game_over:
        return

    text = self.input_box.text.strip()

    if not text:
        self.feedback_msg = "Please enter a valid number first."
        self.feedback_color = (240, 200, 80)
        return

    try:
        guess = int(text)
    except ValueError:
        self.feedback_msg = "Please enter a valid number first."
        self.feedback_color = (240, 200, 80)
        return

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.low_bound = max(self.low_bound, guess + 1)
        self.feedback_msg = f"T00 LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
        result = "T00 LOW"
        result_color = (80, 160, 240)

    elif guess > self.secret_number:
        self.high_bound = min(self.high_bound, guess - 1)
        self.feedback_msg = f"T00 HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
        result = "T00 HIGH"
        result_color = (240, 100, 80)

    else:
```

```

self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
self.feedback_color = (80, 220, 90)
self.game_won = True
result = "CORRECT"
result_color = (80, 220, 90)

self.guess_history.append((guess, result, result_color))

if len(self.guess_history) > 5:
    self.guess_history.pop(0)

# Check for game over after an incorrect guess
if not self.game_won and self.attempts >= self.max_attempts:
    self.game_over = True
    self.feedback_msg = f"GAME OVER! The number was {self.secret_number}."
    self.feedback_color = (240, 80, 80)

```

The important part is that the attempt is incremented **only after the input has been validated**.

3. Reset everything in `reset()`

Update `reset()` :

`</>` Python



▶ Run

```

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.low_bound = 1
    self.high_bound = 100
    self.guess_history = []
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.game_over = False
    self.input_box.clear()

```

So pressing `R` completely starts a fresh game.

4. Change the attempts display

In `render()` , replace:

`</>` Python



▶ Run

```
attempts_surf = self.font_medium.render(  
    f"Attempts: {self.attempts}", True, (180, 185, 195)  
)
```

with:

</> Python



▶ Run

```
attempts_surf = self.font_medium.render(  
    f"Attempts: {self.attempts} / {self.max_attempts}",  
    True,  
    (180, 185, 195)  
)
```

It will now show:

Attempts: 3 / 7



5. Show the restart message for both states

Currently you have:

</> Python



▶ Run

```
if self.game_won:  
    restart_surf = self.font_medium.render(  
        "Press [R] to Start a New Game",  
        True,  
        (255, 220, 80)  
    )  
    screen.blit(  
        restart_surf,  
        (self.width // 2 - restart_surf.get_width() // 2, 295)  
    )
```

Change it to:

</> Python



▶ Run

```
if self.game_won:  
    restart_msg = "Press [R] to Start a New Game"  
elif self.game_over:  
    restart_msg = "Press [R] to Try Again"  
  
if self.game_won or self.game_over:  
    restart_surf = self.font_medium.render(  
        restart_msg,
```

```

        True,
        (255, 220, 80)
    )
    screen.blit(
        restart_surf,
        (self.width // 2 - restart_surf.get_width() // 2, 295)
    )

```

6. Update the R key handling

Your current code only resets when the player has won:

</> Python



▶ Run

```

elif event.key == pygame.K_r and self.game_won:
    self.reset()

```

Change it to:

</> Python



▶ Run

```

elif event.key == pygame.K_r and (self.game_won or self.game_over):
    self.reset()

```

This means R works after either winning **or** running out of attempts.

Final behavior

The game now follows this flow:

```

Attempts: 1 / 7
Current Possible Range: 1 - 100

[ 50 ] [SUBMIT]

T00 HIGH! (Guess was 50)

```



After seven incorrect guesses:

```

Attempts: 7 / 7
Current Possible Range: 24 - 31

GAME OVER! The number was 27.

Press [R] to Try Again

```



The player cannot submit any more guesses after `GAME_OVER`, and pressing **R** resets:

- Secret number
- Attempts → 0 / 7
- Possible range → 1 – 100
- Guess history → empty
- game_won → False
- game_over → False
- Input box → empty

One important detail: **the 7th guess is still processed normally**. If it is correct, the player wins; only an incorrect 7th guess triggers `GAME_OVER` .



ChatGPT can make mistakes. Check important info.